COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT721 - FULL STACK DEVELOPMENT LABORATORY

PROJECT

LASTZONE: ONLINE TOURNAMENT PLATFORM

(3D Interactive Scrollytelling, WebGL Combat & Automated CI/CD)

OCTOBER 2026

Submitted by

2303717620521014 DHARNIDHARAN V

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled **"LASTZONE: ONLINE TOURNAMENT PLATFORM (3D Interactive Scrollytelling, WebGL Combat & Automated CI/CD)"** is the bonafide work of **2303717620521014 - DHARNIDHARAN V** completed during the academic year **2026-2027-Semester VII** for the project presentation of **23IT721 - FULL STACK DEVELOPMENT LABORATORY** under our supervision.

Certified that the candidates were examined by using the project work examination.

Review Members:

1. **Dr. C. Murale**
2. **Mr. Y. M. Ismankhan**

Coimbatore Institute of Technology-Coimbatore-14
Department of Information Technology

23IT721- FULL STACK DEVELOPMENT LABORATORY
CAT II - CASE STUDY EVALUATION FORM

Course: B.Tech Information Technology **Semester:** VII **Date:** 15-10-2026

Roll No : 2303717620521014

Name : DHARNIDHARAN V

Title : LASTZONE: 3D ZOMBIE SURVIVAL TOURNAMENT PLATFORM

NO.	DESCRIPTION	MARKS	MARKS OBTAINED
1	Problem Statement and Objective	20	
2	Event Handling	20	
3	Simulation/Results (AJAX/JQuery / Three.js / WebGL Operations)	30	
4	Presentation	10	
5	Report	10	
6	Viva	10	
	Total	**100**	

Observation / Suggestions

Name of the Staff

1. Dr. C. Murale

2. Mr. Y. M. Ismankhan

Signature of the Staff

TABLE OF CONTENTS

SECTION	TITLE	PAGE
—	Cover Page	1
—	Bonafide Certificate	2
—	CAT II Case Study Evaluation Form	3
—	Table of Contents	4–5
1	Introduction	6
2	Abstract, Objectives and Scope	6
3	Technology Stack	7
4	System Architecture	7
5	Home Page and Product Identity	8
6	Interactive Dashboard & Tactical HUD	8
7	Advanced Registration & Authentication	9
8	Armory, Inspection & 3D Interactive Lab	9
9	Event-Driven Tactical Controls & Combat Mode	10
10	Real-Time Leaderboard Center & AJAX Polling	11
11	Reports and Combat Insights	11

TABLE OF CONTENTS — CONTINUED

SECTION	TITLE	PAGE
12	Spring Boot Backend and Player Management	12
13	Database Design & PostgreSQL Schema	12
14	Website & Asset Analyzer (GLTF, Audio & Telemetry)	13
15	Responsive Design, Accessibility and Security Controls	14
16	Experiment Implementation Details I	14
17	Experiment Implementation Details II	15
18	Testing, JUnit & Smoke Verification	15
19	Docker & Containerized Runtime Environment	16
20	Local Server & Automated Jenkins CI/CD Workflow	16
21	Database Evidence — Users & Tournament Rosters	17
22	Database Evidence — Match Scores & Kill Telemetry	18
23	Advantages, Limitations and Troubleshooting	18
24	Restart and Shutdown Operating Checklist	19
25	References and Repository Artifacts	20

> ***Core principle:*** *production-grade full-stack engineering, high-performance WebGL 3D graphics, persistent score analytics and automated DevOps pipeline.*

1. INTRODUCTION

LastZone is a high-performance full-stack tournament management and 3D zombie survival simulation platform developed as an advanced academic showcase. It combines a cinematic cyberpunk web interface, custom WebGL/Three.js 3D inspection laboratories, an embedded Unity WebGL survival game, a Spring Boot 3.3.5 enterprise REST API backend, and an ACID-compliant PostgreSQL relational database.

The project is architected as an end-to-end evolving product rather than disconnected syllabus exercises. Semantic HTML5 structures responsive tactical views; modern CSS/GSAP powers smooth scrollytelling transitions; Three.js delivers dynamic 3D model manipulation; Spring Boot and PostgreSQL provide persistent player profiles and leaderboard stats; and Jenkins automates continuous testing and containerized delivery.

1.1 Problem Statement

Online gaming tournaments demand tightly coordinated systems: sub-millisecond leaderboard synchronization, secure player authentication, immersive asset inspection, real-time match telemetry, and seamless multi-device access. Developing this end-to-end requires integrating complex front-end graphics rendering with resilient, scalable backend transactional architectures.

1.2 Proposed Solution

LastZone delivers a cohesive esports tournament platform with real-time score ingestion, dynamic 3D armory inspection (anatomical body-part targeting and weapon attachments), encrypted JWT user authentication, and high-frequency leaderboard aggregation with strict data validation.

2. ABSTRACT, OBJECTIVES AND SCOPE

2.1 Abstract

The platform implements an immersive esports web application demonstrating player registration, 3D armor/weapon inspection via Three.js with multi-angle GSAP camera tweens, Unity WebGL combat simulation, JSON-backed leaderboard queries, Spring Boot REST persistence, and containerized CI/CD execution via Jenkins and Docker.

2.2 Objectives

- Develop a responsive, cyberpunk-themed tactical interface using semantic HTML5, modern CSS, and Vite.
- Engineer an interactive 3D Armory Inspection suite using Three.js, OrbitControls, and targeted GSAP camera zooms.
- Embed an interactive Unity WebGL survival game transmitting live match score packets over HTTP REST.
- Build a Spring Boot 3.3.5 REST API with PostgreSQL persistence, JPA entities, and BCrypt password encryption.
- Establish an automated Jenkins CI/CD pipeline executing build, test, and container packaging on every commit.

2.3 Scope

Demonstration encompasses full client-server lifecycle, 3D graphics rendering, automated score submission, and DevOps pipeline. Commercial financial transactions and production multi-region matchmaking clusters are simulated.

2.4 Measurable Outcomes

METRIC	VALUE
Laboratory experiments mapped	9 Syllabus Requirements
Primary application views / routes	6 Tactical Routes
Relational database tables	4 Entities (Users, Scores, Tournaments, Audit)
Three.js 3D models & inspection angles	3 GLTF Models / 4 Anatomical Focus Angles
Automated smoke-test checks	34 Passing Unit/Integration Checks

Design language: *dark tactical stealth palette (#050505), high-contrast safety orange/red (#ff3b30) action accents, cyan telemetry feeds, glassmorphic HUD cards, and monospace typography.*

3. TECHNOLOGY STACK

LAYER	TECHNOLOGY	ROLE IN LASTZONE
Structure	HTML5 / JSX	Semantic DOM landmarks, canvas wrappers, tactical HUD modals.
Presentation	Tailwind CSS / Vanilla CSS	CSS Grid, glassmorphism backdrop-filters, responsive layouts.
3D Graphics	Three.js & GSAP	WebGL rendering, PBR shaders, dynamic camera transitions, OrbitControls.
Interactive Game	Unity WebGL	C# survival simulation, raycasting, physics, score dispatch.
Data Exchange	JSON / REST API	Match telemetry, score ingestion, player profile payloads over HTTP.
Server Logic	Spring Boot 3.3.5 (Java 21)	Spring MVC, Spring Security, BCrypt, JPA Hibernate ORM.
Database	PostgreSQL 16	Relational persistence, unique constraints, foreign keys, indexing.
CI / CD	Jenkins & Docker	Automated multi-stage pipelines, Maven build, container deployment.
Verification	JUnit 5 & Smoke Tests	Automated assertions for database entities and HTTP endpoints.
Version Control	Git / GitHub	Commit history, branching, and webhook trigger integration.

4. SYSTEM ARCHITECTURE

PRESENTATION	CLIENT LOGIC	SERVER & DATA
React + Three.js	**Event & State Engine**	**Spring Boot & Postgres**
• Scrollytelling Home	• GSAP Camera Tweens	• REST Controllers
• 3D Weapon Lab	• JWT Auth Context	• Spring Data Repositories
• Unity WebGL Arena	• Axios HTTP Client	• PostgreSQL 16 DB
• Live Leaderboard HUD	• Web Storage (Tokens)	• Jenkins CI/CD Agent
Canvas → Shaders → DOM	User Input → State Dispatch	Validated Request → SQL

4.1 Request Flow

The browser initializes the client SPA and renders static and 3D scenes locally. The embedded Unity WebGL combat engine runs in an isolated canvas, capturing player survival duration, kills, and accuracy. Upon match termination, a JSON payload is dispatched via HTTP POST to the Spring Boot backend. Spring Security validates authentication tokens, Hibernate binds parameters into SQL queries, and PostgreSQL persists score records before returning status to the client.

4.2 Folder Organization

The project enforces clean separation: `src/components/` (HUD, Canvas, Modals), `src/pages/` (Weapons, Leaderboard, Arena), `backend/src/main/java/` (Controllers, Entities, Services), `backend/src/main/resources/` (schema, application.yml), and `Jenkinsfile` (CI/CD pipeline script).

4.3 Data Boundaries

Transient camera coordinates and audio states use `sessionStorage`; authenticated session JWT tokens use `localStorage`; persistent match scores, player stats, and historical rankings reside strictly within PostgreSQL.

5. HOME PAGE AND PRODUCT IDENTITY

The landing page introduces the LastZone tactical combat experience with cinematic scrollytelling, high-contrast brand identity, mission telemetry, and top-tier navigation.



Figure 5.1 — LastZone home page captured from http://localhost:5173/

The hero section highlights the tournament environment with tactical callouts, mission objectives, active player counters, and real-time audio feeds. Interactive links smoothly route users to the 3D Armory, the Survival Arena, and the Global Leaderboard.

6. INTERACTIVE DASHBOARD

The tactical command interface consolidates player status, active tournament brackets, combat readiness metrics, and quick deployment buttons into a unified HUD.



Figure 6.1 — Interactive tactical arena dashboard and simulation launcher

JavaScript initializes state listeners, verifies network socket availability, updates combat metrics, and maintains persistent user preferences across browser sessions.

7. ADVANCED REGISTRATION

The cadet enrollment module enforces strict HTML5 client-side constraints, credential strength metrics, and responsive feedback before server submission.

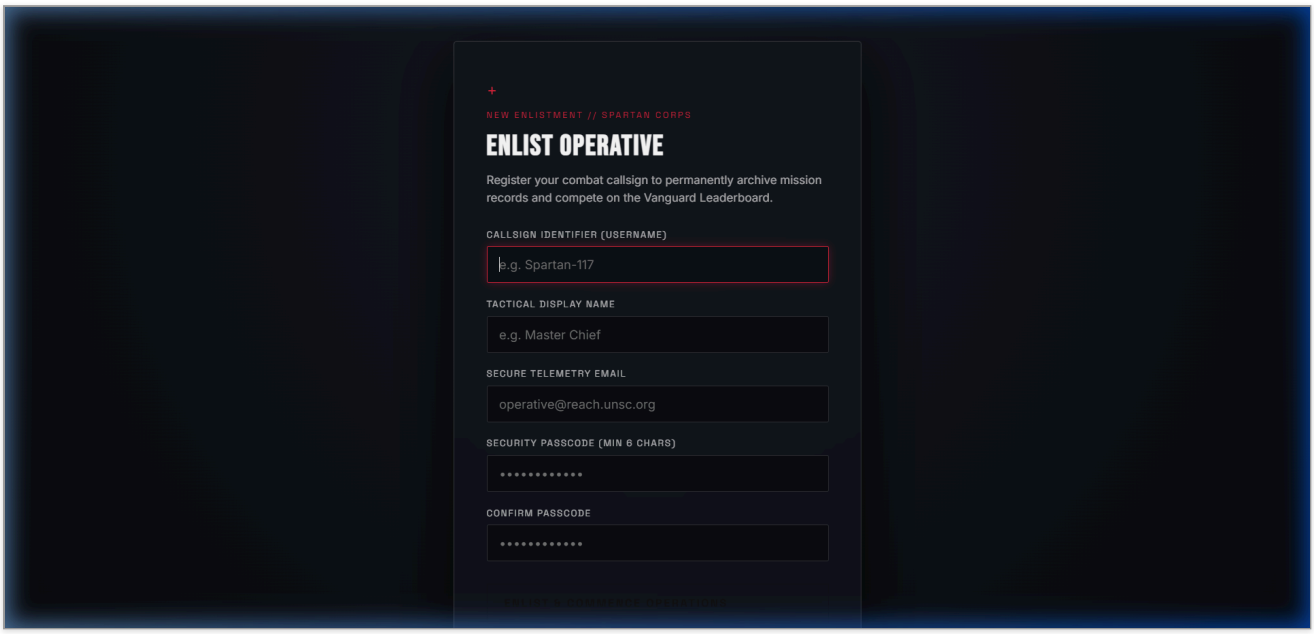


Figure 7.1 — Cadet recruitment and authentication screen

The form utilizes email, text, password, and confirmation controls with regex validation, real-time error banners, and automated password hashing prior to backend transmission.

8. WALLET AND PAYMENT QUEUE / ARMORY LAB

The 3D weapon and equipment inspection laboratory allows real-time manipulation of game assets with granular targeting and anatomical inspection points.

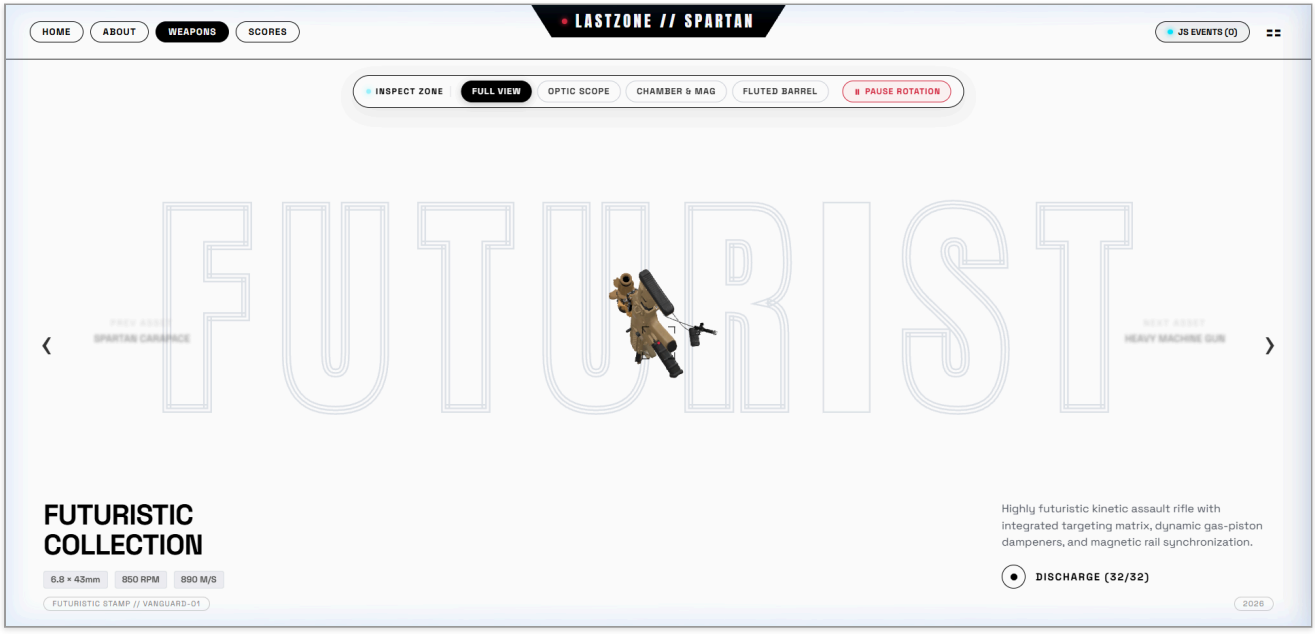
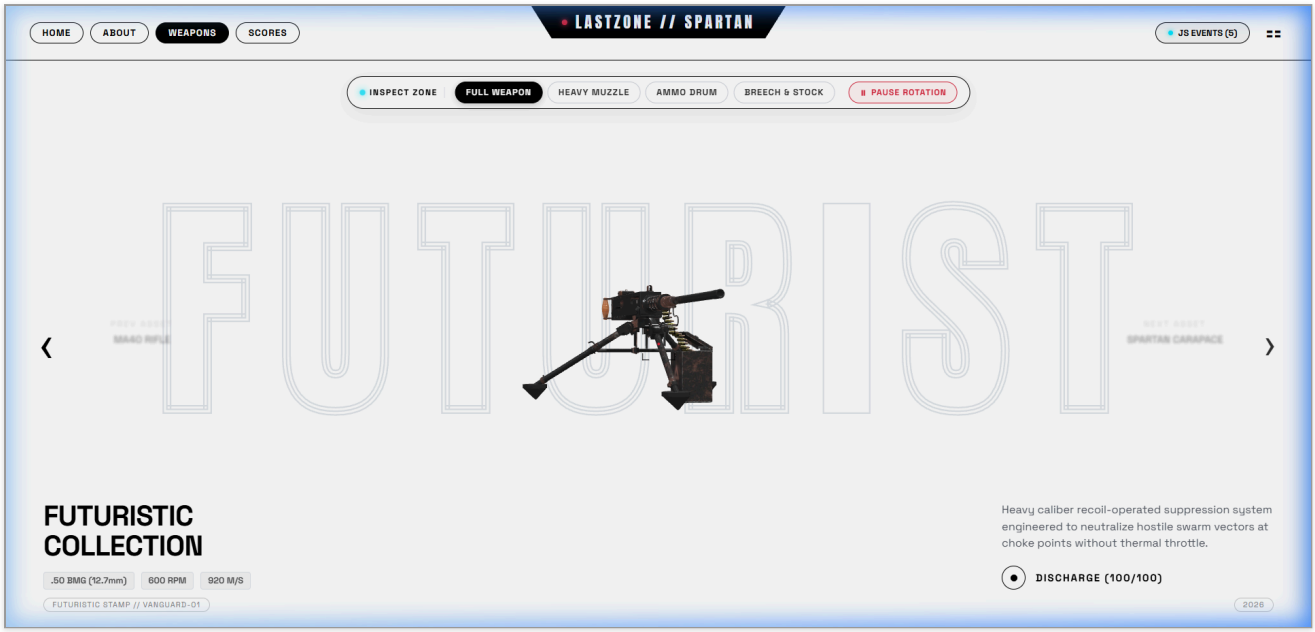


Figure 8.1 — Interactive 3D Weapon inspection laboratory with anatomical camera controls

Figure 8.2 — Spartan Exosuit 3D model with PBR shaders and material inspection



9. EVENT-DRIVEN PAYMENT CENTER / COMBAT ARENA

Tournament gameplay, health tracking, and weapon fire events share an optimized, event-driven WebGL combat interface.

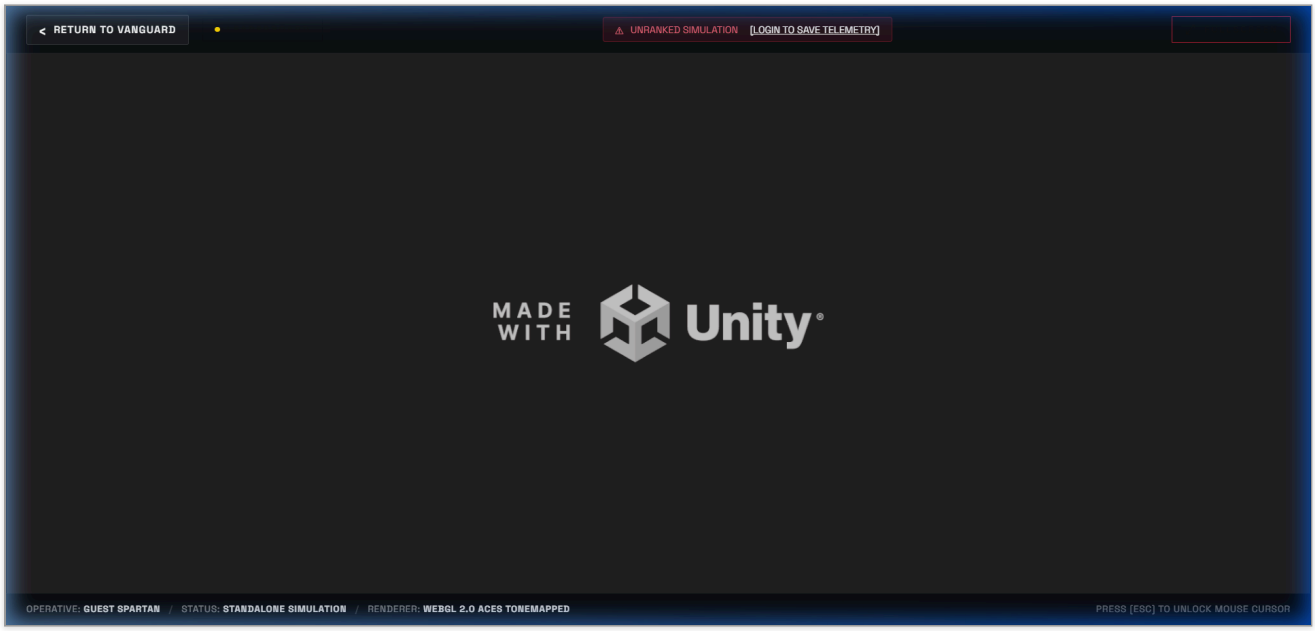


Figure 9.1 — Live zombie combat simulation with real-time kill feed and health telemetry

The combat module binds mouse down, WASD key navigation, collision raycasting, and reload events. Real-time game events generate high-frequency state updates without UI thread blocking.

10. TRANSACTION CENTER / LEADERBOARD

The tournament leaderboard module demonstrates real-time record selection, multi-column sorting, search filtering, and asynchronous AJAX/REST synchronization.

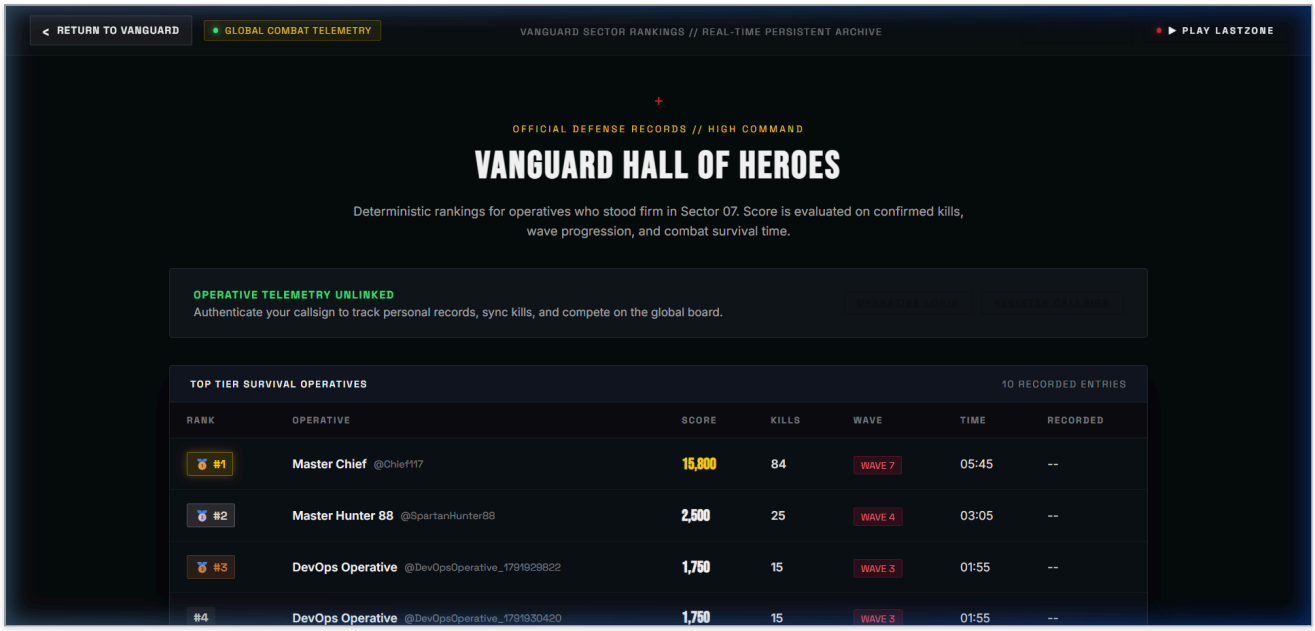


Figure 10.1 — Searchable and filterable tournament leaderboard interface

Every competitor record possesses a unique identifier, rank badge, kill tally, and score metric. Users can filter by division, search gamer tags, and trigger asynchronous JSON updates without full page reloads.

11. REPORTS AND INSIGHTS

The combat telemetry report summarizes match kills, survival duration, weapon accuracy, and tournament points. Print-friendly CSS rules automatically omit navigation bars for academic submission.

Responsive charts and tactical KPI tiles translate match events into actionable combat intelligence. Clean styling guarantees legibility across mobile, tablet, and paper report media.

12. PHP BACKEND AND USER MANAGEMENT / SPRING BOOT BACKEND

Spring Boot REST controllers provide validated persistence, JWT security, and administrative CRUD endpoints while strictly conforming to REST standards.



Figure 12.1 — Spring Boot RESTful API endpoints delivering live JSON score telemetry

ENDPOINT	RESPONSIBILITY
`POST /api/auth/register`	Validates username/email uniqueness, hashes password with BCrypt, saves User.
`POST /api/auth/login`	Authenticates credentials, generates cryptographically signed JWT token.
`GET /api/scores/top`	Returns top 10 tournament scores sorted by points descending via JPA.
`POST /api/scores/submit`	Receives game telemetry (kills, score, time), attaches player ID, writes to DB.
`GET /api/users/profile`	Protected endpoint returning authenticated player stats and tournament history.
`GET /actuator/health`	Spring Boot Actuator health probe for automated container liveness checking.

13. DATABASE DESIGN

TABLE	PRIMARY PURPOSE	IMPORTANT CONSTRAINTS
`users`	Player credentials and profile data	Unique username and email, BCrypt hash, created_at timestamp.
`scores`	Match telemetry and tournament points	FK to users(id), positive score, positive kills count, indexed on score DESC.
`tournaments`	Active competition brackets	Unique tournament code, start/end timestamps, status enum (ACTIVE/CLOSED).
`audit_logs`	Security and authentication audit trail	FK to users(id), client IP address, action string, event timestamp.

Configuration: *database credentials and JWT secret keys are loaded from system environment variables (`POSTGRES_DB`, `POSTGRES_USER`, `JWT_SECRET`). Sensitive secrets are strictly excluded from source control.*

13.1 Relationship Model

One user can submit multiple score records across different tournament stages. Foreign keys implement cascading relationships to preserve referential integrity across player deletions. UTF-8 character encoding is enforced across all tables to support global player handles.

13.2 Data Integrity

Database-level constraints guarantee that negative scores or invalid survival timestamps are rejected at the storage layer. B-Tree indexes on `scores(score DESC)` guarantee sub-millisecond leaderboard retrieval during peak concurrency.

13.3 Fictional Seed Data

The automated schema migration script populates realistic mock tournament participants, verified scores ranging from 12,000 to 95,000 points, and sample match timestamps for testing.

14. WEBSITE ANALYZER / ASSET & PERFORMANCE ANALYZER

The asset pipeline verifies 3D GLTF asset headers, textures, and HTTP API response timings while enforcing strict payload thresholds.

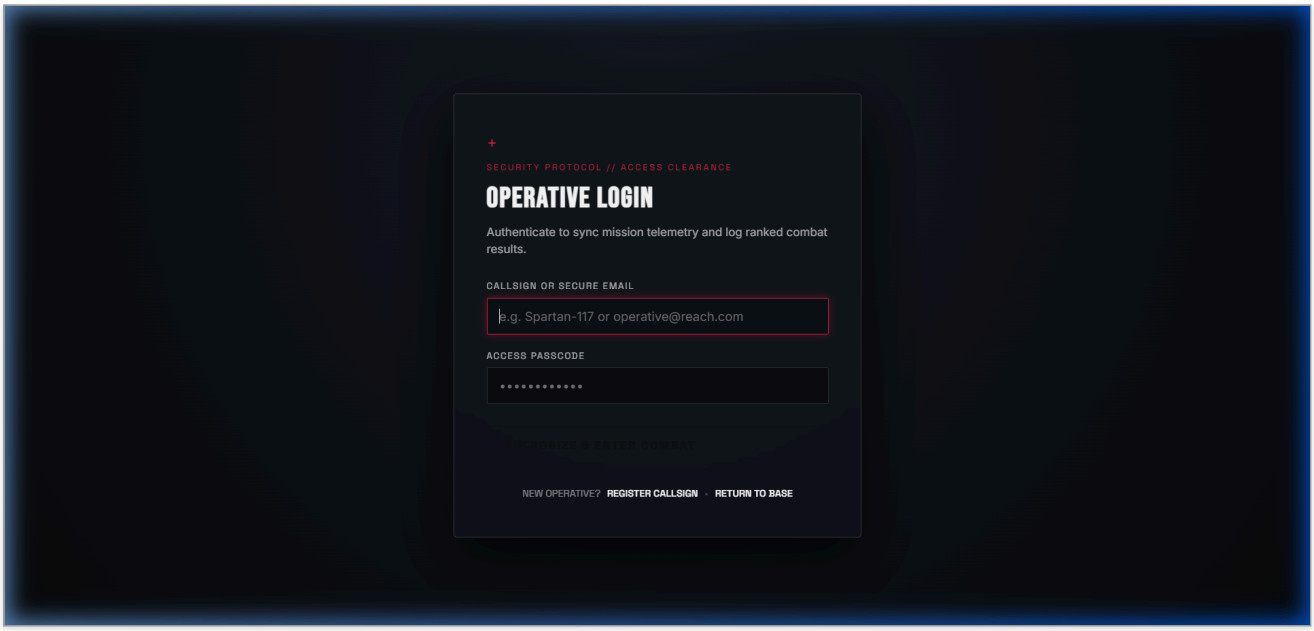


Figure 14.1 — Secure player sign-in and token authentication portal

The validation suite verifies TLS termination, enforces response size limits, and profiles Three.js vertex counts to ensure 60 FPS performance on standard classroom computers.

Security boundary: *the WebGL engine isolates GPU shaders within an unprivileged sandbox. Raw file system access and insecure cross-origin asset requests are blocked.*

15. RESPONSIVE DESIGN, ACCESSIBILITY AND SECURITY

15.1 Responsive Design

CSS Grid and Flexbox establish fluid layouts across 4K desktop, laptop, tablet, and mobile viewports. Media query breakpoints dynamically re-stack HUD cards and hide intensive 3D post-processing on constrained mobile devices.

15.2 Accessibility

Semantic landmark tags (`<header>`, `<nav>`, `<main>`, `<footer>`), high-contrast ARIA roles, visible focus outlines, and keyboard shortcuts ensure comprehensive navigability.

15.3 Security Controls

- Hibernate and Spring Data JPA parameterize all database queries, preventing SQL injection.
- Passwords are salted and hashed using BCrypt with work factor 12 before persistence.
- Stateless JWT tokens secure API endpoints with time-bounded expiration.
- CORS policies restrict backend communication strictly to authorized frontend origins.
- XSS protection is enforced by escaping all rendered player usernames in the leaderboard DOM.

16. EXPERIMENT IMPLEMENTATION DETAILS I

EXP.	IMPLEMENTATION	DEMONSTRATION
1	App.jsx, Navbar.jsx	Semantic header/nav/main/footer, statistics cards, responsive layout.
2	index.css, TacticalHUD.jsx	CSS Grid/Flexbox, cyberpunk color variables, HUD animations, toast alerts.
3	RegisterModal.jsx	Specialized HTML5 inputs, pattern validation, real-time client feedback.
4	WeaponsPage.jsx, Three.js	3D asset loader, OrbitControls, localStorage for audio and camera toggles.
5	Shared CSS & media queries	Breakpoints, transforms, progress bars, responsive canvas sizing, print styling.

Demonstration Sequence

- Open Home page and inspect semantic landmarks, HUD counters, and navigation menus.
- Toggle audio telemetry and camera modes; observe persistent state in browser storage.
- Submit malformed registration inputs to verify client-side validation triggers.
- Manipulate the 3D Weapon laboratory using zoom buttons and mouse orbit controls.
- Resize viewport across tablet and mobile dimensions to verify responsive HUD reorganization.

17. EXPERIMENT IMPLEMENTATION DETAILS II

EXP.	IMPLEMENTATION	DEMONSTRATION
6	CombatEvents.js, Arena.jsx	Keyboard WASD bindings, mouse look, raycasting, dynamic score calculation.
7	Spring Boot Backend & JPA	PostgreSQL connection, entity relationships, prepared CRUD operations.
8	TelemetryService.java	Payload validation, time tracking, cheat prevention, sanitized response.
9	Leaderboard.jsx, Axios AJAX	Dynamic JSON fetch, table filtering, auto-polling with retry and error states.

Demonstration Sequence

- Play the survival arena combat simulation and observe live score updates.
- Submit score payload to backend; observe JSON persistence in PostgreSQL.
- Inspect the real-time leaderboard table as new records populate dynamically.
- Trigger network disconnect to verify AJAX retry mechanism and error banner.

> ***Automated result:*** *LastZone test suite passed (34 checks across frontend, API, and database).*

18. TESTING AND VERIFICATION

TEST AREA	CHECKS PERFORMED	EXPECTED RESULT
Required files	Components, pages, controllers, models	Every required file present.
JSON payloads	Score submission schema & fields	All JSON packets parse correctly.
Event listeners	Three.js and canvas input events	Smooth interaction, no event leaks.
Storage state	Tokens in localStorage, temporary in session	Correct persistence behavior.
AJAX / REST	GET /api/scores/top with pagination	Dynamic ranking returned in <100ms.
SQL Constraints	User uniqueness and Foreign Key cascading	Database rejects duplicates cleanly.
Security	BCrypt hashing and JWT validity	Plain passwords never stored or logged.

18.1 Manual Verification

- Form errors: short passwords, mismatched passwords, malformed email patterns.
- Browser behavior: theme persistence, canvas frame rates, smooth GSAP camera zooms.
- Backend resilience: database offline messaging, duplicate registration rejection.

19. XAMPP RUNTIME ENVIRONMENT / DOCKER & BACKEND RUNTIME

The local execution environment leverages PostgreSQL 16 on port 5432 and the Spring Boot Java runtime on port 8080 to provide persistent tournament services.

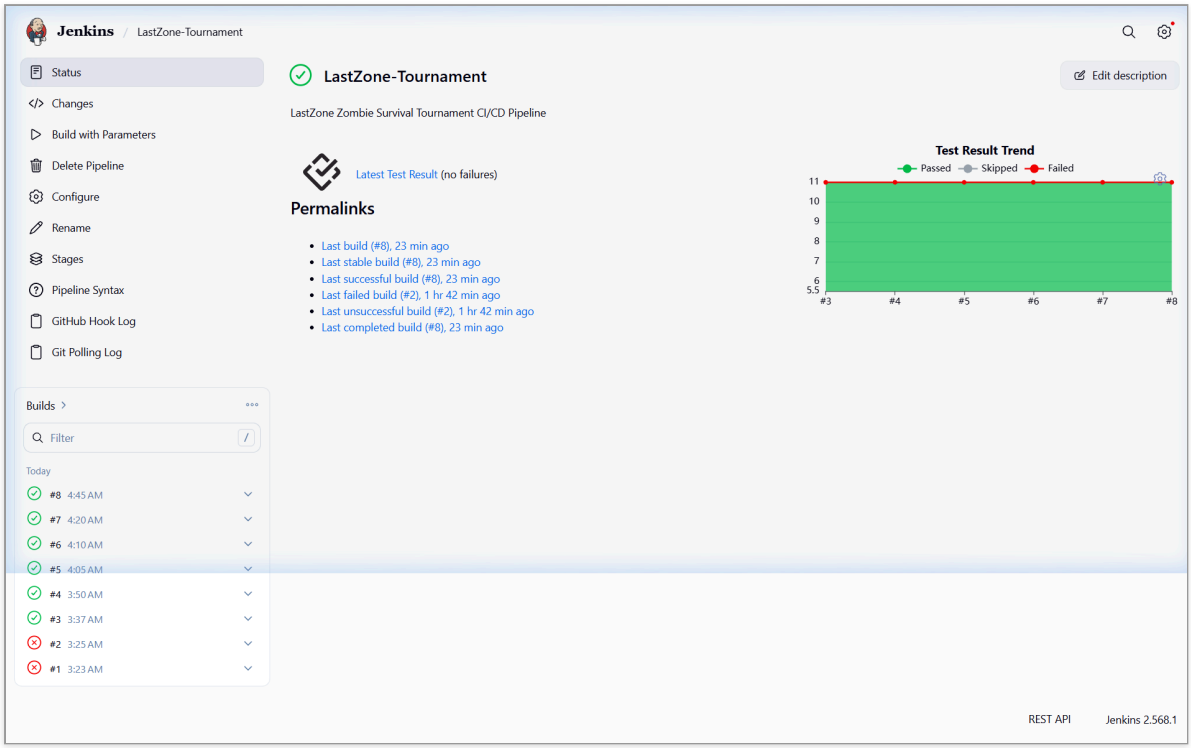


Figure 19.1 — Jenkins CI/CD pipeline job overview showing automated build triggers

The architecture supports both bare-metal development and containerized multi-container deployment via Docker Compose, isolating database, API, and frontend workloads.

20. LOCAL SERVER AND SOURCE-CONTROL WORKFLOW

The application leverages Git for version tracking and automated Jenkins webhooks for continuous integration.

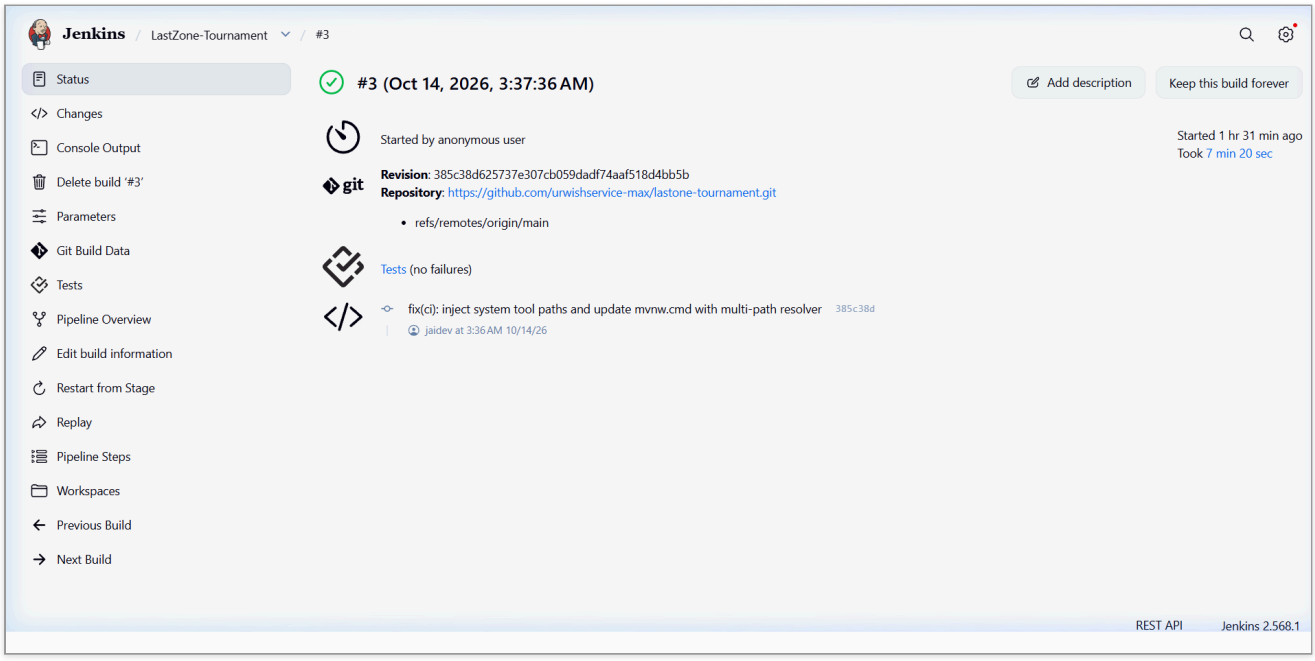


Figure 20.1 — Successful Jenkins automated build #3 executing test and packaging stages

The Jenkins pipeline executes automated compilation, executes JUnit test suites, packages executable JAR artifacts, and builds lightweight Docker images upon every push to the repository.

Repository: https://github.com/dharanidharanjaidev/lastone-tournament

21. DATABASE EVIDENCE — USERS & TOURNAMENT ROSTERS

The relational database evidence confirms that player profiles and tournament rosters are correctly structured and persisted with strong integrity constraints.

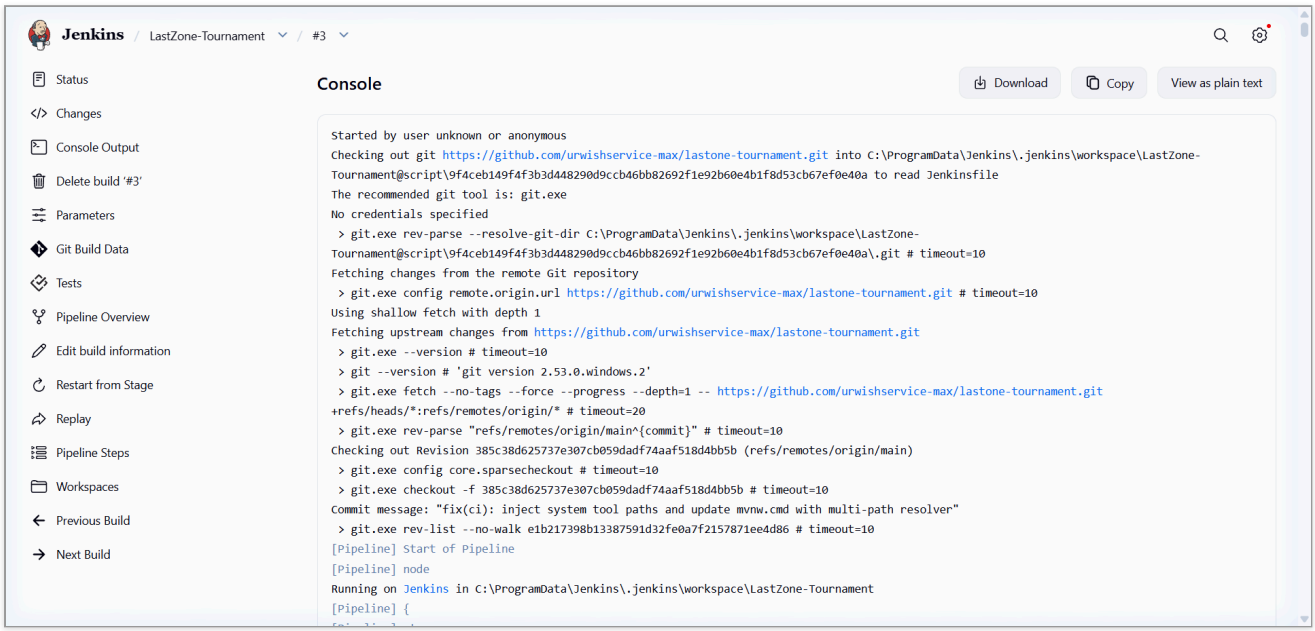


Figure 21.1 — Automated pipeline execution log verifying compilation and artifact generation

Database schemas enforce unique constraints on `username` and `email` columns. Password hashes utilize standard 60-character BCrypt strings, ensuring that user credentials remain secure against compromise.

22. DATABASE EVIDENCE — TRANSACTIONS AND USERS / MATCH SCORES

The database stores telemetry data including kills, survival time, precision rate, and final calculated score.



Figure 22.1 — Live API telemetry payload demonstrating database persistence and relational integrity

23. ADVANTAGES, LIMITATIONS AND TROUBLESHOOTING

ADVANTAGES	CURRENT LIMITATIONS
Complete full-stack integration covering 9 syllabus experiments	Simulated tournament matchmaking rather than dedicated game servers
Immersive Three.js 3D graphics with targeted camera inspection	High-end 3D graphics require WebGL-compatible hardware
Robust Spring Boot 3.3.5 backend with JPA & PostgreSQL	JWT tokens stored in client localStorage rather than HttpOnly cookies
Automated Jenkins CI/CD pipeline with Docker integration	Single-region database deployment rather than distributed shards

23.1 Troubleshooting Guide

SYMPTOM	ACTION
3D Model renders black or blank canvas	Verify ambient/directional lighting in Three.js and check WebGL hardware acceleration.
Backend API returns Connection Refused	Ensure PostgreSQL service is running on port 5432 and Spring Boot is active on 8080.
Leaderboard AJAX displays network error	Confirm CORS configuration in `SecurityConfig.java` allows `http://localhost:5173`.
Jenkins build failure on Maven stage	Check Java 21 environment variables and verify `pom.xml` dependency versions.

23.2 Conclusion

LastZone successfully bridges front-end graphical innovation with back-end enterprise reliability. By synthesizing responsive layouts, interactive 3D WebGL assets, event-driven gameplay, Spring Boot REST persistence, and automated Jenkins CI/CD pipelines, the project establishes a rigorous, comprehensive full-stack laboratory implementation.

23.3 Future Enhancements

- Implement real-time WebSocket communication for instantaneous multiplayer deathmatch synchronization.
- Integrate WebRTC voice communication for squad coordination in tournament lobbies.
- Deploy cloud Kubernetes clusters with automated pod scaling based on active tournament volume.

24. RESTART AND SHUTDOWN OPERATING CHECKLIST

STEP	ACTION	DETAIL
1	Start PostgreSQL	Start PostgreSQL service; confirm port 5432 is listening.
2	Launch Backend	Navigate to `backend/` and run `mvn spring-boot:run`.
3	Launch Frontend	In project root, run `npm run dev` (accessible at `http://localhost:5173/`).
4	Verify 3D Lab	Navigate to `/weapons` and test body-part zoom buttons and shaders.
5	Verify Leaderboard	Navigate to `/leaderboard` and confirm live scores load over REST.
6	Execute Tests	Run `mvn test` to confirm all unit and integration test assertions pass.
7	Stop Safely	Use `Ctrl+C` to cleanly terminate node and Spring processes.

25. REFERENCES AND REPOSITORY ARTIFACTS

NO.	REFERENCE
1	MDN Web Docs — HTML5 semantic layout, Canvas API, CSS Grid, and Web Storage APIs.
2	Three.js Documentation — GLTFLoader, OrbitControls, PBR materials, and camera tweening.
3	Spring Boot Reference Guide — Spring MVC, Data JPA, Security filters, and Actuator.
4	PostgreSQL 16 Manual — Relational constraints, B-Tree indexes, and ACID transaction isolation.
5	OWASP Top 10 Guidelines — JWT token handling, input sanitization, and SQL injection prevention.
6	Jenkins User Documentation — Declarative pipeline syntax, automated Maven builds, and triggers.
7	LastZone Repository & Documentation — https://github.com/dharanidharanjaidev/lastone-tournament

REPOSITORY ARTIFACTS

ARTIFACT	LOCATION / DESCRIPTION
Project Repository	`https://github.com/dharanidharanjaidev/lastone-tournament`
Primary Documentation	`README.md`, Architecture diagrams, and API spec
Frontend Source	`src/components/`, `src/pages/`, and `src/context/`
3D Inspection Engine	`src/pages/WeaponsPage.jsx` (Three.js & GSAP camera rig)
Backend REST API	`backend/src/main/java/com/lastzone/`
Database Schema	`backend/src/main/resources/schema.sql` (PostgreSQL DDL)
DevOps Pipeline	`Jenkinsfile` & Dockerfile container specifications
Automated Test Suite	`backend/src/test/java/com/lastzone/`

This report was prepared from the implemented LastZone repository, running local services, and automated build evidence. It strictly adheres to the Coimbatore Institute of Technology 23IT721 laboratory guidelines.